

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 6

Inside the xv6 File System

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Read What xv6 Shows You 2
2.1.1 Observe Answers (Part 1) 3
2.2 Part 2: Write `mystat` 4
2.2.1 Observe Answers (Part 2) 4
2.3 Part 3: Measure File System Limits 5
2.3.1 Observe Answers (Part 3) 5
2.4 Part 4: Write `find` 7

3 CONCLUSIONS **8**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The main purpose of this laboratory exercise is to analyze the internal structure of the xv6 file system. It involves the creation of user-space programs in C (such as `mystat` and a recursive `find`), gaining insight into how hard links are managed at the kernel level, and evaluating the structural constraints established in `kernel/fs.h`.

1.2 Theoretical Background

The xv6 file system is designed with a layered Unix-like architecture:

- **Disk Layer & Superblock:** Specifies the total number of blocks on the disk, allocates blocks for the log, and determines the count of inode blocks.
- **Inode Layer:** On the disk, each file is represented by a `struct dinode`, which contains metadata (such as type, size, and link count) along with block pointers (12 direct blocks and 1 indirect block).
- **Directory Layer:** Directories function as special files that store arrays of `struct dirent` entries (each having a fixed size of 16 bytes: 2 bytes for the inode number and a 14-byte array for the name, defined as `DIRSIZ`).
- **System Calls:** The `stat()` call fetches metadata into a `struct stat`, whereas `sys_unlink()` reduces the hard link counters, freeing up data blocks when the link count reaches zero.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Read What xv6 Shows You

When executing `ls` in xv6, the output displays entries in the format `name, type, inode, size`.

```
hart 2 starting
hart 1 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
mystat         2 23 36344
find           2 24 42720
console        3 25 0
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
$
```

Figure 2.1: Inspection of default xv6 directory entries and types

2.1.1 Observe Answers (Part 1)

1. **Type Codes (`kernel/stat.h`):**
   - `1` = `T_DIR` (Directory)
   - `2` = `T_FILE` (Regular File)
   - `3` = `T_DEVICE` (Device, such as a console)
2. **Root Directory Inode:** The root directory `/` is assigned the inode number **1**. Both the `.` and `..` entries share this same inode number (`1`) because the root directory lacks a parent directory; thus, `..` refers to itself.
3. **Directory Entry Size:** Every `struct dirent` takes up $2 + 14 = 16$ bytes. An empty directory, like `a`, holds 2 entries (`.` and `..`), which amounts to a total size of $2 \times 16 = 32$ bytes.

2.2 Part 2: Write `mystat`

I developed a custom tool, `user/mystat.c`, to display the file metadata obtained through the `stat()` function.

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 2.2: Execution of mystat with hard link creation and deletion

2.2.1 Observe Answers (Part 2)

1. **Hard Link Output Changes:**
   - Following the command `ln README r2`: The link count for inode 2 went up from 1 to 2. Both the file size (2441 bytes) and the inode number stayed the same for both file names.
   - Following the command `rm README`: The link count for `r2` dropped from 2 to 1. The contents of the file continued to be fully accessible.
2. **Unlink Implementation in Kernel:** The statement `ip->nlink-;` located in `kernel/sysfile.c` is responsible for reducing the link count. Data blocks are subsequently released within the `iput()` function once `nlink == 0` and there are no active processes referencing it.
3. `struct dinode` **Fields (**`kernel/fs.h`**):** This structure includes fields such as `type`, `major`, `minor`, `nlink`, `size`, and an array `addrs[NDIRECT+1]`.

2.3 Part 3: Measure File System Limits

The physical boundaries of the disk are defined by system constants found in `kernel/fs.h`.



Figure 2.3: Inspection of kernel limits, disk layout, and filename truncation

2.3.1 Observe Answers (Part 3)

1. **Filename Truncation:** The command `mkdir averyveryverylongname` results in the directory being named `averyveryverylo` due to the limitation set by `#define DIRSIZ 14`, which restricts filenames to a maximum of 14 bytes.

2. **Maximum File Size Calculation:**

$$\text{Direct Blocks} = 12 \times 1024 = 12,288 \text{ bytes (12 KB)}$$
$$\text{Indirect Blocks} = \frac{1024}{4} = 256 \text{ blocks} \rightarrow 256 \times 1024 = 262,144 \text{ bytes (256 KB)}$$
$$\text{Max Size} = (12 + 256) \times 1024 = 274,432 \text{ bytes} (\approx 268 \text{ KB})$$

3. **Disk Layout Metadata:** Out of a total of 2000 blocks (2 MB), metadata consumes 46 blocks (comprising 1 boot block, 1 superblock, 30 log blocks, 13 inode blocks, and 1 bitmap block), leaving 1954 blocks available for data storage.

2.4 Part 4: Write `find`

The program `user/find.c` is designed to recursively search through directory structures, deliberately ignoring the `.` and `..` references.

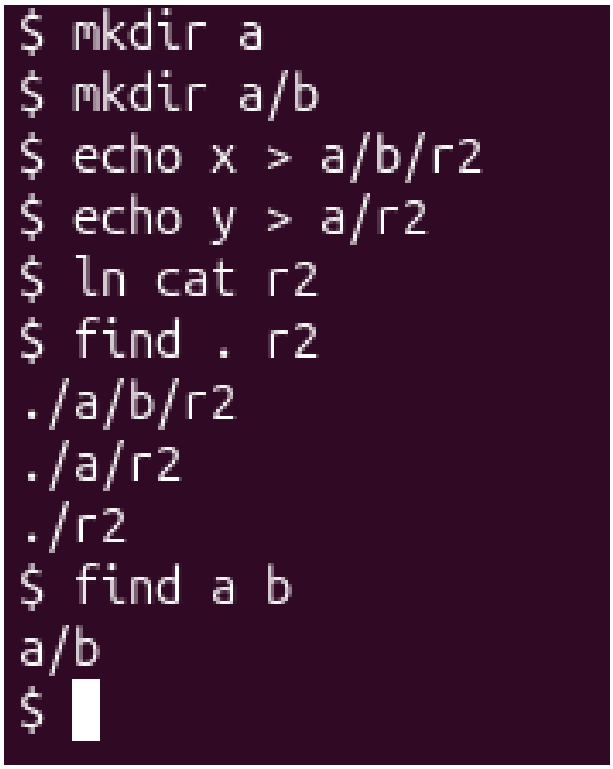


Figure 2.4: Recursive search execution with find utility

3 CONCLUSIONS

Throughout this laboratory session, I investigated the inner workings of the xv6 file system. By developing `mystat.c`, I acquired hands-on experience with how system calls retrieve fundamental inode metadata and understood how hard links govern the lifespan of a file, irrespective of its name.

By examining `kernel/fs.h`, I was able to determine the precise mathematical constraints of the xv6 file system structure, such as the limits on filename length (`DIRSIZ`) and the maximum file sizes allowed by single indirect block addressing. Lastly, creating the recursive `find.c` tool solidified my understanding of directory traversal techniques, string manipulation, and the importance of resource management (like closing descriptors) within a constrained operating system environment.